National Autonomous University of Mexico

Faculty of Engineering

Compilers

Group 5

Parser & SDT

Syntax and Semantic Analyzer

Technical Report

Team 07

Account number	Role
321193692	Presenter
321203018	Technical Writer
321135344	Developer
321237125	Model Designer
321132422	Project Manager

Repository: https://github.com/HansZuniga/Team-07-Parser-SDT

Package: `unam.fi.compilers.g5.07`

November 10, 2026

Contents

1 Introduction **2**

2 Project objective **2**

3 Problem definition **3**

4 Supported C language scope **3**

5 Project architecture **5**

6 Lexical analyzer integration **5**

7 CLR(1) theoretical background **7**
7.1 Bottom-up parsing . . . 7
7.2 LR(1) items . . . 7
7.3 Why CLR(1) . . . 8
7.4 CLOSURE and GOTO . . . 8
7.5 Construction of the ACTION/GOTO table . . . 9
7.6 The LR parsing algorithm . . . 9

8 Grammar definition **9**
8.1 Design decisions of the grammar . . . 10
8.2 The professor's example . . . 11

9 Augmented grammar **11**

10 Canonical collection of LR(1) items **12**

11 ACTION/GOTO parsing table **13**

12 Parser implementation **14**
12.1 Line of a syntax error . . . 15

13 Parse tree construction **15**

14 SDT rules **16**
14.1 Type policy . . . 17

15 Semantic analysis **19**
15.1 Example: from the tree to the result . . . 19

16 Symbol table **20**

17 Error handling **20**

18 Test cases **21**

19 Results **21**
19.1 Representative executions . . . 21
19.2 Theoretical justification of each output . . . 23
19.3 Justification of each test case . . . 24

20 Conclusions **26**

References **27**

1 Introduction

A compiler is organized in phases. The **lexical analyzer** turns the characters of a source program into tokens; the **syntax analyzer** (*parser*) checks that the sequence of tokens follows the grammar of the language and builds a tree that shows its structure; the **semantic analyzer** uses that tree to check properties that a context-free grammar cannot express, such as whether a variable was declared or whether a value of one type can be stored in a variable of another type [1].

In the previous project, Team 07 built a lexical analyzer in Python for a subset of C [7]. This report documents the second part: a **bottom-up CLR(1) parser** and a **Syntax-Directed Translation (SDT)** stage that performs the semantic analysis. Both were built only with the theoretical concepts seen in class: the canonical collection of LR(1) items, the CLOSURE and GOTO operations and the ACTION/GOTO table were computed by the team, and the parsing driver, the parse tree, the symbol table and the semantic rules were written from scratch, without parser generators or parsing libraries.

The document first defines the problem and the supported language, then presents the theory of CLR(1) parsing and the grammar, the canonical collection and the parsing table built for it. After that it describes the implementation (lexer integration, parser driver, parse tree, SDT rules and symbol table) and, finally, the test cases and the theoretical justification of every output the program can produce.

2 Project objective

General objective. Build a Parser & SDT that receives the tokens produced by the team's lexical analyzer, verifies them against a context-free grammar using a bottom-up CLR(1) parser, constructs the parse tree and validates it with SDT rules, reporting `Parsing Success!` / `SDT Verified!` or the corresponding error with its line number.

Specific objectives.

- Reuse the lexer of the first project as the only source of input tokens.
- Define a grammar whose terminals are exactly the tokens of the lexer, covering a broad subset of C.
- Build, by hand and with the methods seen in class, the canonical collection of LR(1) items and the CLR(1) ACTION/GOTO table, and show that the table has no conflicts.
- Implement the CLR(1) driver (shift, reduce, goto, accept) and build the parse tree during the reductions.
- Define SDT rules for every production, with a symbol table that handles scopes and types, and verify them on the parse tree.
- Distinguish lexical, syntax and semantic errors, including the case in which parsing succeeds but SDT validation fails, and report the line.
- Validate the whole system with automated tests and justify every result theoretically.

3 Problem definition

The project statement [4] asks for a program that reads a file or a string and applies the following procedure:

1. Scan the input (lexical analysis) to obtain the tokens.
2. Parse the tokens with a grammar whose productions match them, using a top-down or bottom-up method, and construct the parse tree.
3. Verify the parse tree against SDT rules (semantic analysis).
4. If the input ends in a syntactically valid state and the SDT rules are satisfied, the process is successful; otherwise it ends in an error, reporting the line (`error in line n`).

The statement also requires considering the case where parsing is successful but SDT validation fails. Together with the lexical error inherited from the first project (if the input contains a character sequence that is not a valid token, there is no valid token stream to parse), the program has the four possible outcomes summarized in Table 1.

Table 1: Possible outcomes of the program.

Situation	**Output**	**Phase that decides it**
Valid syntax and valid semantics	`Parsing Success!` + `SDT Verified!`	SDT
Valid syntax, invalid semantics	`Parsing Success!` + `SDT error in line n`	SDT
Invalid syntax	`Parsing error in line n`	Parser
Invalid token	`Lexical error in line n`	Lexer

The professor's reference input is `int x = 10;`, with the expected output `Parsing Success!` and `SDT Verified!`. Section 8.2 explains how this example is handled by the team's grammar.

4 Supported C language scope

The analyzed language is a subset of C whose unit of analysis is a complete program formed by the function `int main()` and its block. Table 2 lists what is and what is not supported.

String literals are recognized by the grammar so that an input such as `int x = "hello";` is *syntactically* valid and can be rejected by the semantic analysis, which is exactly the "parsing success but SDT error" case required by the statement.

Table 2: Supported subset of C.

Construct	**Supported forms**
Program	`int main() { ... }`
Types	`int`, `float`, `char`
Declarations	One variable per declaration, with or without initializer: `int x;`, `float y = x + 2.5;`
Statements	Assignment `x = expr;`, `return expr;`, empty statement `;`, nested blocks `{ ... }`
Control flow	`if`, `if/else` (including the *dangling else*), `while`, with or without braces
Expressions	`== != < <= > >= + - * / %`, prefix `+ - !` and parentheses, with the usual C precedence and associativity
Literals	Integer (`10`), decimal (`2.5`), character (`'a'`) and string (`"hello"`)
Not supported	Other functions and function calls, global declarations, arrays, pointers, `for`, `switch`, `struct`, preprocessor directives, compound assignment (`+=`) and logical operators `&& ||`.

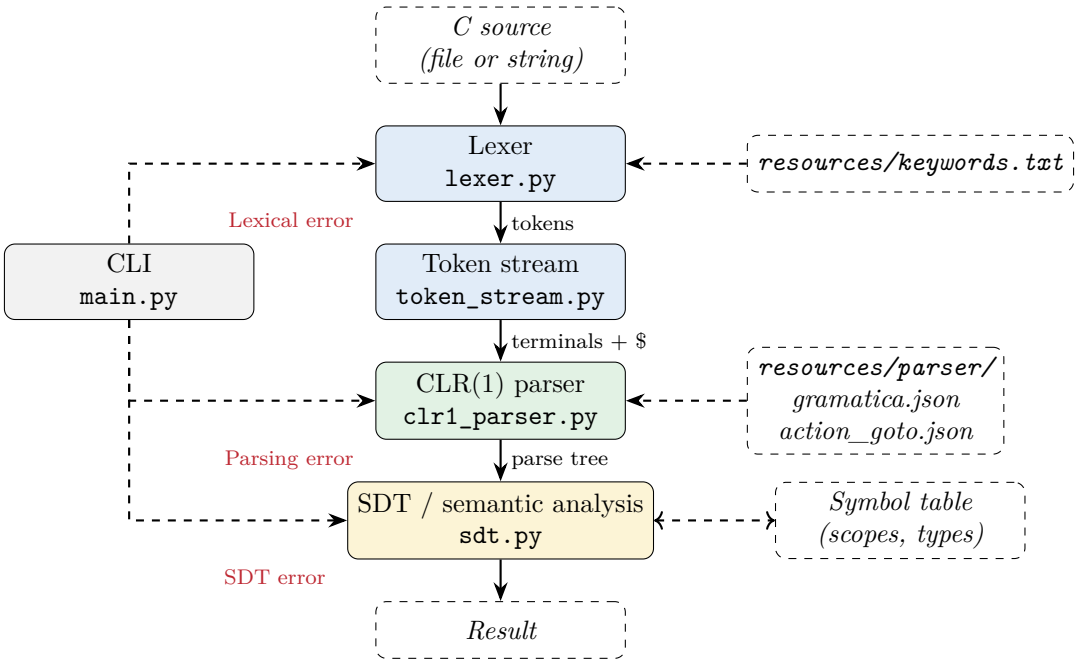


Figure 1: Architecture of the project. Each phase stops the pipeline when it finds an error, so a later phase only runs on a valid input.

5 Project architecture

The application follows the package structure `mx/unam/fi/compilers/g5/07` requested by the statement and separates each phase into its own module [8], so that each one can be tested on its own (Figure 1).

```
07/
|-- resources/
|   |-- keywords.txt          # print, int, float, char, return, if, else, while
|   `-- parser/               # gramatica.json + action_goto.json (Model Designer)
|-- src/main/
|   |-- lexer.py              # lexer of the previous project (+ float/char/while)
|   |-- token_stream.py       # bridge lexer -> parser: tokens to terminals + $
|   |-- clr1_parser.py        # CLR(1) engine: shift/reduce/goto/accept, tree
|   |-- sdt.py                # SDT: symbol table, scopes and types
|   `-- main.py               # command line interface
|-- examples/
`-- tests/                    # pytest + tests/cases (42 programs)
```

The grammar and the parsing table are *data*: the parser reads them from JSON files instead of hard-coding them. This keeps the theoretical model of the Model Designer and the implementation consistent: the same productions 0–50 and states 0–160 appear in the documents, in the JSON files and in the traces printed by the program.

6 Lexical analyzer integration

As the statement requires, the input of the parser is the output of a lexical analyzer. The parser never reads characters: `token_stream.py` runs the lexer of the first project and translates each token into a *terminal* of the grammar. For this project the lexer was extended with the keywords `float`, `char` and `while`, character literals (`'c'`) and the operators `%` and `!`.

The translation rules are shown in Table 3. Keywords, operators and punctuation use their own lexeme as terminal; identifiers become `ID`; constants and literals become `INT`, `FLOAT`, `CHAR` or `STRING`. The identifier `main` is still an identifier for the lexer (in C it is not a reserved word), but it is mapped to the terminal `main` because the grammar expects `int main ( ) Block`. Finally, the end-of-input marker `$` is appended exactly once.

Table 3: Translation from lexer tokens to grammar terminals.

Lexer token	Example	Terminal
KEYWORD	int, while	the lexeme: int, while
IDENTIFIER	main	main
IDENTIFIER	x, suma	ID
CONSTANT	10 / 3.5	INT / FLOAT
LITERAL	'a' / "hola"	CHAR / STRING
OPERATOR	=, <=	the lexeme: =, <=
PUNCTUATION	;, {	the lexeme: ;, {
end of input	–	$

Every terminal keeps a reference to its original token (lexeme, line and column). That is what allows the parser and the SDT to report *the line* of an error. If the lexer reports any error, `token_stream.py` raises `LexicalError` and the parser is never called: an invalid token stream is never treated as valid parser input. Figure 2 shows the flow for one declaration, and Figures 3 and 4 show the real output of both stages for the same program.

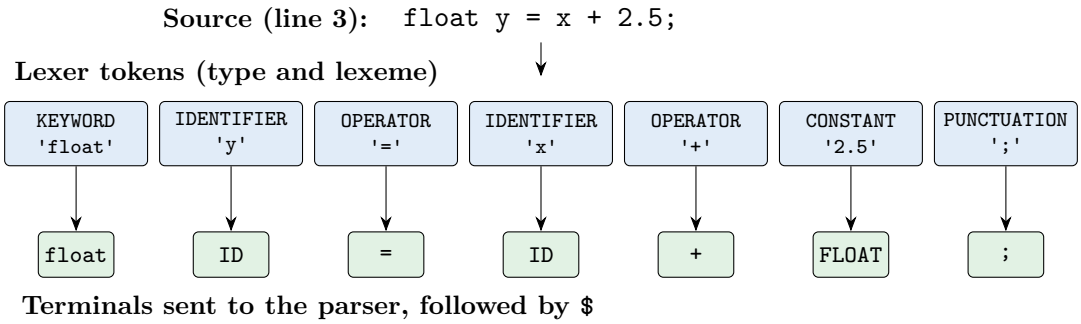


Figure 2: Lexer-to-parser token flow: each token keeps its lexeme, line and column, but the parser only looks at the terminal symbol.

```
Terminal — Team 07 Parser & SDT

07/src/main$ python main.py --file ../../tests/cases/valid/08_document_example.c
TYPE         LEXEME                    LINE   COLUMN
-----------------------------------------------------
KEYWORD      'int'                     1      1
IDENTIFIER   'main'                    1      5
PUNCTUATION  '('                       1      9
PUNCTUATION  ')'                       1      10
PUNCTUATION  '{'                       1      12
KEYWORD      'int'                     2      5
IDENTIFIER   'x'                       2      9
OPERATOR     '='                       2      11
CONSTANT     '10'                      2      13
PUNCTUATION  ';'                       2      15
KEYWORD      'float'                   3      5
IDENTIFIER   'y'                       3      11
OPERATOR     '='                       3      13
IDENTIFIER   'x'                       3      15
OPERATOR     '+'                       3      17
CONSTANT     '2.5'                     3      19
PUNCTUATION  ';'                       3      22
KEYWORD      'return'                  4      5
IDENTIFIER   'x'                       4      12
PUNCTUATION  ';'                       4      13
PUNCTUATION  '}'                       5      1

Total of tokens: 21
```

Figure 3: Tokens produced by the lexer for `valid/08_document_example.c`.

```
Terminal — Team 07 Parser & SDT
07/src/main$ python main.py --file ../../tests/cases/valid/08_document_example.c --terminals
TERMINAL     LEXEME                  LINE   COLUMN
-----------------------------------------------------
int          'int'                   1      1
main         'main'                  1      5
(            '('                     1      9
)            ')'                     1      10
{            '{'                     1      12
int          'int'                   2      5
ID           'x'                     2      9
=            '='                     2      11
INT          '10'                    2      13
;            ';'                     2      15
float        'float'                 3      5
ID           'y'                     3      11
=            '='                     3      13
ID           'x'                     3      15
+            '+'                     3      17
FLOAT        '2.5'                   3      19
;            ';'                     3      22
return       'return'                4      5
ID           'x'                     4      12
;            ';'                     4      13
}            '}'                     5      1
$            '$'                     5      2

Parser input: int main ( ) { int ID = INT ; float ID = ID + FLOAT ; return ID ; } $
```

Figure 4: The same program translated into grammar terminals; the last line is the exact input that the CLR(1) parser receives.

7 CLR(1) theoretical background

7.1 Bottom-up parsing

A bottom-up parser builds the parse tree from the leaves (the tokens) up to the root (the start symbol). It is a *shift-reduce* parser: it keeps a stack, **shifts** input symbols onto it and, when the top of the stack matches the right-hand side of a production (a *handle*), **reduces** it to the left-hand side. Read in reverse, the sequence of reductions is a rightmost derivation of the input [1].

The LR family of parsers (*L*eft-to-right scan, *R*ightmost derivation in reverse) decides when to shift and when to reduce by means of a deterministic finite automaton whose states are sets of *items*. The decisions are stored in two tables:

- **ACTION**$[i,\ a]$, for a state i and a terminal a: *shift* j, *reduce* by production k, *accept*, or *error* (empty cell).
- **GOTO**$[i,\ A]$, for a state i and a nonterminal A: the state reached after reducing to A.

7.2 LR(1) items

An LR(0) item is a production with a dot in its right-hand side, such as $A \to \alpha \cdot \beta$: α has already been recognized and β is expected next. An **LR(1) item** adds a *lookahead* terminal:

$$[A \to \alpha \cdot \beta,\ a]$$

which means "we are recognizing $A \to \alpha\beta$, and after A the next input symbol must be a". When the dot reaches the end, $[A \to \alpha\cdot, a]$ authorizes the reduction by $A \to \alpha$ *only* when the next input symbol is a.

7.3 Why CLR(1)

The methods seen in class differ in *where* they place the reductions:

- **LR(0)** reduces in every column of the row, regardless of the next symbol. Almost any realistic grammar has conflicts.
- **SLR(1)** reduces $A \to \alpha$ on every terminal of FOLLOW(A). FOLLOW is global for the whole grammar, so it can authorize reductions in contexts where they are not valid, which produces shift/reduce and reduce/reduce conflicts.
- **CLR(1)** (canonical LR(1)) reduces only on the lookahead of the specific item, computed in its context. It is the most powerful of these methods: every SLR(1) and LALR(1) grammar is CLR(1). LR(k) parsing was introduced by Knuth [3].
- **LALR(1)** merges CLR(1) states with the same LR(0) core; it has fewer states but can introduce reduce/reduce conflicts.

The team chose **CLR(1)** because the goal was to cover a broad and realistic subset of C: nested statements, the *dangling else*, and an expression grammar with six precedence levels. The extra precision of the lookaheads means that the grammar can be written naturally (left recursion, ε-productions) without conflicts. A concrete example appears in the collection: states I_9 and I_{122} have the same core `Simple -> ;` ., but only I_{122} has `else` as a lookahead. In CLR(1) they are kept as different states, so the reduction by p24 is authorized before `else` only where it is valid. The price of CLR(1) is a larger number of states (161), which is not a problem because the table is computed once and stored as data.

7.4 CLOSURE and GOTO

The canonical collection is built with two operations [1]:

CLOSURE(I). If $[A \to \alpha \cdot B\beta,\ a]$ is in I and $B \to \gamma$ is a production, add $[B \to \cdot\gamma,\ b]$ for every terminal $b \in \text{FIRST}(\beta a)$. Repeat until no new item can be added. The lookahead b is computed from what follows B *in this item* (β), and only if β can derive ε does the original lookahead a propagate.

GOTO(I, X). Move the dot over the symbol X in every item of I that has X right after the dot, and apply CLOSURE to the result.

The collection starts with I_0 = CLOSURE({[`Start` → ·`Program`, \$]}) and applies GOTO with every grammar symbol to every state until no new set of items appears.

Example in the team's collection (state I_{28}). After reading `int x` inside a block, the parser is in I_{28}, which contains

$$[\texttt{Decl} \to \texttt{Type ID} \cdot \texttt{Init ;},\ a].$$

The symbol after the dot is the nonterminal `Init` and the rest of the item is $\beta =$ `;`, so FIRST(βa) = FIRST(`;` a) = {`;`}. CLOSURE therefore adds both productions of `Init` with lookahead `;`:

$$[\texttt{Init} \to \cdot\ \texttt{= Expr},\ \texttt{;}] \qquad [\texttt{Init} \to \cdot,\ \texttt{;}]$$

The first item lets the parser shift `=` (GOTO(I_{28}, =) = I_{49}). The second is already complete, so it authorizes the reduction `Init ->` ε (p12) *only* before `;`. That is exactly what happens in `int x;`. After the reduction, GOTO(I_{28}, Init) = I_{50} contains [`Decl` → `Type ID Init` · `;`, a].

7.5 Construction of the ACTION/GOTO table

For every state I_i of the collection:

1. If GOTO(I_i, t) = I_j and t is a terminal, ACTION[i, t] = shift j.
2. If [$A \to \alpha\cdot,\ a$] $\in I_i$ and $A \neq$ `Start`, ACTION[i, a] = reduce by $A \to \alpha$ (only in column a).
3. If [`Start` $\to$ `Program`$\cdot$, \$] $\in I_i$, ACTION[i, \$] = accept.
4. If GOTO(I_i, A) = I_j and A is a nonterminal, GOTO[i, A] = j.

A cell that receives two different actions is a *conflict*; the grammar is CLR(1) if and only if no cell has a conflict.

7.6 The LR parsing algorithm

With the table, the driver is the same for every LR method:

```
push state 0;  a = first terminal
loop:
    s = state on top of the stack
    case ACTION[s, a] of
      shift j:   push j;  a = next terminal
      reduce k:  (A -> beta) = production k
                 pop |beta| states;  t = new top
                 push GOTO[t, A];  build node A with the popped children
      accept:    return the tree (Parsing Success!)
      empty:     syntax error in the line of a
```

A reduce does not consume input, and a reduce by an ε-production pops nothing; it only pushes the GOTO state.

8 Grammar definition

A context-free grammar is a 4-tuple $G = (N, T, P, S)$ of nonterminals, terminals, productions and start symbol [2]. The grammar of this project was designed by the Model Designer on top of the tokens of the lexer [5].

Terminals ($|T| = 31$, plus `$` as the 32nd ACTION column):

```
!  != % ( ) * + - / ; < <= = == > >= CHAR FLOAT ID INT STRING
      char else float if int main return while { }
```

Nonterminals ($|N| = 18$): `Program`, `Block`, `Items`, `Item`, `Decl`, `Type`, `Init`, `Stmt`, `Matched`, `Unmatched`, `Simple`, `Expr`, `Equality`, `Relation`, `Add`, `Mul`, `Unary`, `Primary`.

Start symbol: `Program`.

Productions: the 51 productions are listed in Table 4. Their numbers are the ones used by the parsing table, the reductions of the trace and the SDT rules.

Table 4: Productions of the grammar (p0 is the augmentation).

#	LHS	RHS	#	LHS	RHS
p0	Start	Program	p26	Equality	Equality == Relation
p1	Program	int main ( ) Block	p27	Equality	Equality != Relation
p2	Block	{ Items }	p28	Equality	Relation
p3	Items	Items Item	p29	Relation	Relation < Add
p4	Items	ε	p30	Relation	Relation <= Add
p5	Item	Decl	p31	Relation	Relation > Add
p6	Item	Stmt	p32	Relation	Relation >= Add
p7	Decl	Type ID Init ;	p33	Relation	Add
p8	Type	int	p34	Add	Add + Mul
p9	Type	float	p35	Add	Add - Mul
p10	Type	char	p36	Add	Mul
p11	Init	= Expr	p37	Mul	Mul * Unary
p12	Init	ε	p38	Mul	Mul / Unary
p13	Stmt	Matched	p39	Mul	Mul % Unary
p14	Stmt	Unmatched	p40	Mul	Unary
p15	Matched	Simple	p41	Unary	+ Unary
p16	Matched	Block	p42	Unary	- Unary
p17	Matched	if (Expr) Matched else Matched	p43	Unary	! Unary
p18	Matched	while (Expr) Matched	p44	Unary	Primary
p19	Unmatched	if (Expr) Stmt	p45	Primary	ID
p20	Unmatched	if (Expr) Matched else Unmatched	p46	Primary	INT
p21	Unmatched	while (Expr) Unmatched	p47	Primary	FLOAT
p22	Simple	ID = Expr ;	p48	Primary	CHAR
p23	Simple	return Expr ;	p49	Primary	STRING
p24	Simple	;	p50	Primary	(Expr)
p25	Expr	Equality			

8.1 Design decisions of the grammar

- **Program and blocks.** `Program -> int main ( ) Block` and `Block -> { Items }`. `Items` is a left-recursive list that can be empty (p3, p4). Left recursion is natural for LR parsers: the list is reduced as soon as each item ends, so the stack does not grow with the length of the program.
- **Declarations vs. assignments.** A declaration begins with `Type` (`Decl -> Type ID Init ;`), an assignment begins with `ID` (`Simple -> ID = Expr ;`). The optional initializer is `Init -> = Expr | ` ε.
- **Dangling else.** Statements are split into `Matched` (every `if` has its `else`) and `Unmatched` (some `if` is still open). Only a `Matched` statement can appear between `if (...)` and `else`, so every `else` is attached to the nearest open `if`. The ambiguity is removed by the grammar itself, not by giving priority to shift in the table.
- **Precedence and associativity.** One nonterminal per level, from lowest to highest: `Equality` (`== !=`), `Relation` (`< <= > >=`), `Add` (`+ -`), `Mul` (`* / %`), `Unary` (prefix `+ - !`)

and `Primary`. Binary levels are left-recursive (left associative): `8 - 3 - 1` is `(8 - 3) - 1`; prefix operators are right-recursive. Thus `1 + 2 * 3` is `1 + (2 * 3)` because `Add` uses a whole `Mul` as its right operand.

- **Assignment is a statement.** The assignment is not part of `Expr`, which keeps `=` out of expressions.

8.2 The professor's example

The unit recognized by `Program` is a complete `main` function. The declaration `int x = 10;` is generated by the grammar, but inside a block: `Decl -> Type ID Init ;` with `Type -> int`, `Init -> = Expr` and the chain `Expr` $\Rightarrow^*$ `Primary -> INT`. On its own it is not a `Program`: in state I_0 the only valid action is *shift* `int`, which leads to I_2, where the parser requires `main`. The terminal after `int` is `ID`, so ACTION[2, ID] is empty and the result is `Parsing error in line 1`. Placed inside `main`, which is how it appears in a real C program, the same declaration produces `Parsing Success!` and `SDT Verified!` (Figure 5).

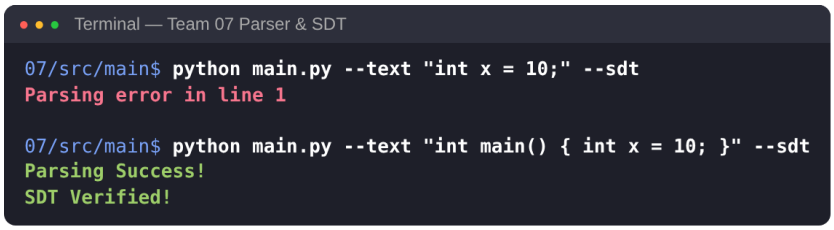


Figure 5: The professor's example alone and inside `int main()`.

9 Augmented grammar

To know exactly when the input has been completely recognized, the grammar is *augmented* with a new start symbol and the production

$$\text{p0:} \quad \texttt{Start} \rightarrow \texttt{Program}$$

The parser accepts when, with the item [`Start` $\rightarrow$ `Program`$\cdot$, \$] on top of the stack, it reads `$`. Without this production the parser could not tell the reduction to `Program` at the end of the input from any other reduction. p0 is never used as an ordinary reduction: its complete item generates ACCEPT in ACTION[1, \$], and `Start` has no column in the GOTO table, which therefore uses only the 18 original nonterminals.

The FIRST sets used by CLOSURE are listed in Table 5.

Table 5: FIRST sets of the nonterminals.

Nonterminal(s)	FIRST
`Start, Program`	`int`
`Block`	`{`
`Items`	`; ID char float if int return while {` ε
`Item`	`; ID char float if int return while {`
`Decl, Type`	`char float int`
`Init`	`=` ε
`Stmt, Matched`	`; ID if return while {`
`Unmatched`	`if while`
`Simple`	`; ID return`
`Expr, Equality, Relation, Add, Mul, Unary`	`! ( + - CHAR FLOAT ID INT STRING`
`Primary`	`( CHAR FLOAT ID INT STRING`

10 Canonical collection of LR(1) items

Applying CLOSURE and GOTO (Section 7.4) from I_0 = CLOSURE({[`Start` → ·`Program`, $]}) produces **161 distinct states** (I_0–I_{160}) and **730 transitions**. No two states are merged, even if they share the same LR(0) core, because that is the definition of the canonical collection. The complete collection is in the Model Designer document [5] and in `docs/model-designer/Datos/coleccion_lr1.json`. Some representative states are:

```
I0 : [Start -> . Program , $]
     [Program -> . int main ( ) Block , $]
I1 : [Start -> Program . , $]                                    (accept)
I5 : [Program -> int main ( ) . Block , $]
     [Block -> . { Items } , $]
I7 : [Block -> { . Items } , $]
     [Items -> . Items Item , ; ID char float if int return while { }]
     [Items -> . , ; ID char float if int return while { }]
I17: [Decl -> Type . ID Init ; , ; ID char float if int return while { }]
I22: [Type -> int . , ID]
I28: [Decl -> Type ID . Init ; , ; ID char float if int return while { }]
     [Init -> . = Expr , ;]
     [Init -> . , ;]
I50: [Decl -> Type ID Init . ; , ; ID char float if int return while { }]
I87: [Decl -> Type ID Init ; . , ; ID char float if int return while { }]
```

In I_7 the lookaheads of `Items` are the FIRST of what can follow a list inside a block: the start of another `Item` or the closing brace `}`. In I_{22} the only lookahead of `Type -> int .` is `ID`, because after a type the grammar always expects the name of the variable. Figure 6 shows the path of the automaton that recognizes `int main() { int x; }` and `int main() { int x = ... }`.

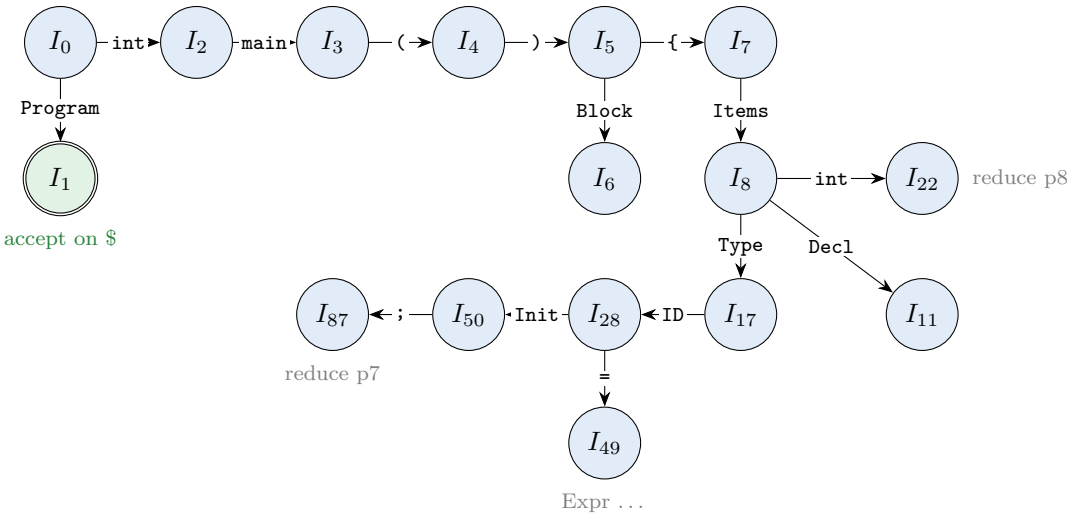


Figure 6: Fragment of the CLR(1) automaton (states and transitions taken from the team's collection).

11 ACTION/GOTO parsing table

Applying the construction rules to the 161 states gives **509 shift actions, 795 reduce actions, 1 accept and 221 GOTO entries**, with **zero shift/reduce and zero reduce/reduce conflicts**. This proves that the grammar is CLR(1). The parser double-checks this property when it loads the table: if a cell contains more than one action, the table is rejected. ACTION has 32 columns (31 terminals and `$`); GOTO has 18 columns.

Table 6 shows the rows used by the examples of this report. Each entry `action: terminals` means that the action applies to every listed column; any column not listed is an error.

Table 6: Excerpt of the ACTION/GOTO table (complete table in [5]).

State	ACTION	GOTO
0	`s2: int`	`Program→1`
1	`acc: $`	-
2	`s3: main`	-
5	`s7: {`	`Block→6`
6	`r1: $`	-
7	`r4: ; ID char float if int return while { }`	`Items→8`
8	`s9: ; | s12: ID | s19: char | s20: float | s21: if | s22: int | s23: return | s24: while | s25: { | s26: }`	`Block→10, Decl→11, Item→13, Matched→14, Simple→15, Stmt→16, Type→17, Unmatched→18`
17	`s28: ID`	-
22	`r8: ID`	-
28	`r12: ; | s49: =`	`Init→50`
49	`s30: ! | s31: ( | s32: + | s33: - | s35: CHAR | s38: FLOAT | s39: ID | s40: INT | s44: STRING`	`Add→34, Equality→36, Expr→86, Mul→41, Primary→42, Relation→43, Unary→45`

State	ACTION	GOTO
50	`s87:  ;`	-
87	`r7:  ; ID char float if int return while { }`	-

How to read some cells:

- ACTION[0, int] = s2: the only way to start a program is the keyword `int`. Any other first token is a syntax error in line 1.
- ACTION[7, t] = r4 for the ten terminals that can start an item or close the block: the empty list `Items ->` ε is reduced without consuming the token.
- ACTION[28, ;] = r12 and ACTION[28, =] = s49: after `Type ID`, a `;` means "no initializer" and `=` starts one. Any other token (for example `ID` in `int x y;`) is an error.
- ACTION[1, $] = acc: the whole input is a `Program`.

12 Parser implementation

The parser is in `clr1_parser.py`. It does not compute the collection at run time: `ParsingTable` loads `gramatica.json` (productions `{id, lhs, rhs}`) and `action_goto.json` (`["s", n]`, `["r", p]` or `["acc", 0]`), checks that no cell has two actions, and builds two dictionaries indexed by (state, symbol).

`Parser.parse` is the LR algorithm of the previous section. It keeps two parallel stacks: one of *states* and one of *tree nodes*.

- **Shift** j: push j and a leaf node with the terminal (it keeps lexeme, line and column), and advance the input.
- **Reduce** k ($A \to \beta$): look at the state exposed under the top $|\beta|$ states, consult GOTO for A, pop $|\beta|$ states and nodes, and push the GOTO state and a new node A whose children are the popped nodes, in order. If $\beta = \varepsilon$, nothing is popped and the node has no children.
- **Accept**: return the root of the tree and print `Parsing Success!`.
- **Empty cell**: raise `ParseError` with the line of the error and the list of terminals that were expected in that state.

With the option `--trace` the parser records the stack, the lookahead and the action of every step. Figure 7 shows the complete execution for `int main() { int x; }`. It matches the theory step by step: the empty list (r4) and the empty initializer (r12) are reduced without consuming the token; `Type -> int` is reduced as soon as `ID` appears (the only lookahead of I_{22}); the reduction by p7 pops four states (`Type ID Init ;`); and the input is accepted with the stack `0 1` and the token `$`.

Terminal — Team 07 Parser & SDT

```
07/src/main$ python main.py --text "int main() { int x; }" --trace
STEP   STACK (states)              INPUT    ACTION
-------------------------------------------------------------------------------
1      0                           int      shift 2
2      0 2                         main     shift 3
3      0 2 3                       (        shift 4
4      0 2 3 4                     )        shift 5
5      0 2 3 4 5                   {        shift 7
6      0 2 3 4 5 7                 int      reduce 4 (Items -> epsilon), goto 8
7      0 2 3 4 5 7 8               int      shift 22
8      0 2 3 4 5 7 8 22            ID       reduce 8 (Type -> int), goto 17
9      0 2 3 4 5 7 8 17            ID       shift 28
10     0 2 3 4 5 7 8 17 28         ;        reduce 12 (Init -> epsilon), goto 50
11     0 2 3 4 5 7 8 17 28 50      ;        shift 87
12     0 2 3 4 5 7 8 17 28 50 87   }        reduce 7 (Decl -> Type ID Init ;), goto 11
13     0 2 3 4 5 7 8 11            }        reduce 5 (Item -> Decl), goto 13
14     0 2 3 4 5 7 8 13            }        reduce 3 (Items -> Items Item), goto 8
15     0 2 3 4 5 7 8               }        shift 26
16     0 2 3 4 5 7 8 26            $        reduce 2 (Block -> { Items }), goto 6
17     0 2 3 4 5 6                 $        reduce 1 (Program -> int main ( ) Block), goto 1
18     0 1                         $        accept

Parsing Success!
```

Figure 7: Parser execution: complete shift/reduce trace.

12.1 Line of a syntax error

An LR parser detects an error at the first token that cannot continue a valid prefix. For most errors, that token is where the problem is. The exception is a missing `;` at the end of a line: the parser only notices it when it reads the first token of the *next* line. To report the line the programmer expects, when the unexpected token starts a new line the parser simulates inserting a `;` before it (without changing the real stacks). If the `;` is accepted and the unexpected token can follow it, the missing `;` was the cause and the error is reported in the line of the previous token; otherwise it is reported in the line of the unexpected token.

13 Parse tree construction

The tree is built bottom-up during the reductions, so it is a direct record of the derivation: every internal node is a nonterminal together with the production used to reduce it, and every leaf is a token. The root is `Program`. Intermediate productions (`Expr -> Equality`, `Equality -> Relation`, ...) are *not* removed: each one is a node, so the SDT rule of every production can be applied to its own node. Figure 8 shows the subtree of the declaration `int x = 10;`; Figure 9 shows the tree printed by the program for a complete example.

The tree also shows the effect of the grammar on precedence: in `x + 2.5` the node `Add` has the children `Add + Mul`; in the test `1 + 2 * 3` the multiplication is a `Mul` node *below* the `+`, so it is evaluated first.

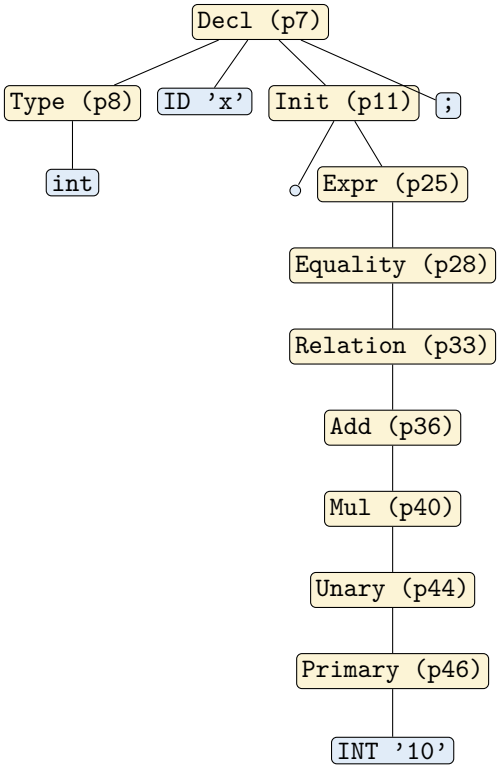


Figure 8: Parse subtree of `int x = 10;` (yellow: nonterminals with their production; blue: leaves/tokens).

14 SDT rules

A Syntax-Directed Translation associates semantic rules with the productions of the grammar; the rules compute *attributes* of the grammar symbols [1]. The project uses mainly *synthesized* attributes (they flow from the children to the parent, the natural direction for a bottom-up parser) and an environment (the symbol table) that is passed through the tree in source order:

- Expressions: `data_type` (`int`, `float`, `char` or `string`) and, for a simple literal, `literal_lexeme`.
- `Type`: `data_type`. `Init`: `present` and the type of its expression.
- `Block`: `scope_id`.

Table 7 lists the semantic action of every production.

Table 7: SDT rules per production.

Prod.	Production	Semantic action
0	`Start -> Program`	ACCEPT; `Program` is the root of the tree.
1	`Program -> int main ( ) Block`	`return_type` := int; visit `Block`.
2	`Block -> { Items }`	Open a new scope (`scope_id`), visit `Items`, close the scope.
3, 4	`Items -> Items Item | ε`	Visit the items from left to right; the empty list is valid.
5, 6	`Item -> Decl | Stmt`	Visit the child.

Prod.	Production	Semantic action
7	`Decl -> Type ID Init ;`	If `ID` exists in the current scope: SEM001. Insert (`ID`, `Type.data_type`) in the table. If `Init.present` and not assignable(`Type`, `Init`): SEM004.
8, 9, 10	`Type -> int | float | char`	`Type.data_type` := int / float / char.
11	`Init -> = Expr`	`Init.present` := true; `Init.type` := `Expr.data_type`.
12	`Init ->` ε	`Init.present` := false.
13–16	`Stmt`, `Matched` chains	Visit the child; a `Block` opens its own scope.
17, 19, 20	`if ( Expr ) ...`	`Expr` must be numeric (else SEM006); visit the branches in source order.
18, 21	`while ( Expr ) ...`	`Expr` must be numeric (else SEM006); check the body once (it is not executed).
22	`Simple -> ID = Expr ;`	Look up `ID` (else SEM002); if not assignable(`ID.type`, `Expr.type`): SEM003.
23	`Simple -> return Expr ;`	`Expr.type` must be assignable to int, the return type of `main` (else SEM007).
24	`Simple -> ;`	No action.
25, 28, 33, 36, 40, 44	pass-through (`Expr -> Equality`, ...)	Copy `data_type` (and `literal_lexeme`) from the only child.
26, 27, 29–32	`== != < <= > >=`	Both operands numeric (else SEM005); result int.
34, 35, 37, 38	`+ - * /`	Both operands numeric (else SEM005); `char` is promoted to int; result float if any operand is float, int otherwise.
39	`Mul -> Mul % Unary`	Both operands char or int (else SEM005); result int.
41, 42	`Unary -> + Unary | - Unary`	Operand numeric (else SEM005); result promote(type).
43	`Unary -> ! Unary`	Operand numeric (else SEM005); result int.
45	`Primary -> ID`	Look up the visible declaration (else SEM002); `data_type` := its type.
46–49	`Primary -> INT | FLOAT | CHAR | STRING`	`data_type` := int / float / char / string; keep the lexeme.
50	`Primary -> ( Expr )`	Copy the attributes of `Expr`.

14.1 Type policy

For initialization, assignment and `return`, a value can be stored in a type of the same or higher rank (*widening* `char -> int -> float`); *narrowing* and strings are rejected (Table 8). This is a simplified, safer policy defined by the team; it is stricter than ISO C, which allows implicit narrowing.

Table 8: assignable(destination, source).

Destination \ source	char	int	float	string
char	yes	no	no	no
int	yes	yes	no	no
float	yes	yes	yes	no



Figure 9: Parse tree printed with `--tree` for `valid/08_document_example.c`.

15 Semantic analysis

The semantic analysis (`sdt.py`) runs **only after** the parser accepts, on the complete tree. `SemanticAnalyzer` visits the tree in source order and, at each node, executes the action of its production (Table 7). It checks names, scopes and types; it does not execute the program (it does not compute values or iterate loops). The first semantic error stops the analysis and is reported with the line of the token responsible for it (for example, the `ID` of a duplicate declaration or the keyword of a non-numeric condition). The errors are classified with the codes of Table 9.

Table 9: Semantic error codes.

Code	Error	Example
SEM001	Duplicate declaration in the same scope	`int x; int x;`
SEM002	Undeclared identifier (or out of scope)	`y = 1;`
SEM003	Incompatible assignment	`char c; c = c + 1;`
SEM004	Incompatible initialization (invalid value for the declared type)	`int x = "hello";`
SEM005	Invalid operand type for an operator	`float f; int m = f % 2;`
SEM006	Non-numeric condition in `if`/`while`	`while ("a") ;`
SEM007	`return` not compatible with int	`return 2.5;`

Why this is semantic and not syntactic. A context-free grammar cannot express "a name must be declared before it is used" or "a name cannot be declared twice in the same block": these are context-sensitive properties, since they relate two distant parts of the program. Types are similar: `int x = "hello";` has exactly the same structure (`Type ID = Expr ;`) as `int x = 10;`; only the attribute `data_type` of the expression distinguishes them. That is why these checks belong to the SDT and why an input can produce `Parsing Success!` followed by `SDT error`.

15.1 Example: from the tree to the result

For the program

```
1 int main() {
2     int x = 10;
3     float y = x + 2.5;
4     return x;
5 }
```

1. p1 fixes the return type of `main` as int; p2 opens scope 0.
2. Line 2, p7: `Type.data_type` = int; `x` is not in scope 0, so it is inserted. `Init` has the literal `10` (p46: int). assignable(int, int) holds.
3. Line 3, p7: `y` is inserted as float. In `x + 2.5`, p45 finds `x`: int; p47 gives float; p34 gives float because one operand is float. assignable(float, float) holds.
4. Line 4, p23: `x` is int, assignable to the return type int.
5. p2 closes scope 0. No rule failed: `SDT Verified!`.

16 Symbol table

The symbol table (`SymbolTable` in `sdt.py`) stores one record per declaration: name, type, scope id, line and column of the declaration, whether it has an initializer, and the initializer's type and lexeme. It keeps a stack of active scopes:

- every `Block` opens a new scope (the block of `main` is scope 0) and closes it when it ends;
- a lookup goes from the innermost active scope to the outermost one;
- declaring a name twice in the same scope is SEM001, but declaring it again in an inner scope is valid (*shadowing*);
- a name declared inside a block is not visible after the block closes (SEM002 if it is used);
- the records of closed scopes are kept, so the complete table can be shown at the end.

The variable is inserted *before* its initializer is checked. This choice is documented by the team: `int x = x;` is accepted, because the analysis checks names and types but not definite initialization.

Figure 10 shows the table for the example above. The initializer of `y` is an expression, so no single lexeme is stored.



Figure 10: Symbol table printed with `--symbols`.

For `valid/06_scopes.c`, the table contains `x` (int, scope 0), `x` (float, scope 1, the shadowing declaration) and `y` (int, scope 2). The statement `return x;` after the inner block finds the `x` of scope 0, of type int, and is therefore valid.

17 Error handling

Each phase stops the pipeline at its first error, and a later phase never reports success if an earlier one failed. The order is:

1. **Lexical error** → `Lexical error in line n`. The lexer still scans the whole input and collects all errors, but the line reported is the first one. The parser is not executed, because the token stream is not valid.
2. **Syntax error** → `Parsing error in line n`. Produced by an empty ACTION cell. The SDT is not executed, since there is no complete tree on which to evaluate the rules. The line is the line of the unexpected token, or of the previous token if a `;` was missing at the end of a line.
3. **Semantic error** → `Parsing Success!` followed by `SDT error in line n`. Parsing succeeded (the tree exists), but an SDT rule failed. Only the first semantic error is reported.

Internally, each error is an exception with additional details (`ParseError.details()` names the expected terminals; `SemanticError.details()` gives the code, column and lexeme), which were used during development and in the tests.

18 Test cases

The project has two levels of tests, both automated with pytest [6]:

- **Unit tests** (208 tests) for each module: the lexer and its extensions, the token-to-terminal translation, the parser engine (shift/reduce/goto/accept on a small grammar, trace, error line, rejection of tables with conflicts, operator precedence in the tree; the shift/reduce trace with the team table, the parse tree of every valid case and the professor's example) and the SDT (compatibility matrix, promotion, scopes, shadowing, every error code, symbol table contents).
- **End-to-end cases** (`tests/cases/`): 42 C programs executed with the real program (`main.py --file <case> --sdt`) and compared with a `.expected` file, in the four categories of the project: 10 valid, 12 syntax errors, 13 semantic errors and 7 lexical errors. `test_cases.py` runs them inside pytest (48 tests: the 42 cases plus checks of the case files), and `run_cases.py` prints the comparison and regenerates `RESULTS.md` with expected and actual outputs.

```
C:\Proyectos\Team-07-Parser-SDT\mx\unam\fi\compilers\g5\07>python -m pytest tests
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Proyectos\Team-07-Parser-SDT\mx\unam\fi\compilers\g5\07
collected 256 items

tests\test_cases.py ..............................................                                    [ 18%]
tests\test_clr1_parser.py ........................................                                    [ 33%]
tests\test_lexer.py ..............................                                                    [ 44%]
tests\test_lexer_extensions.py ...............................                                        [ 57%]
tests\test_parser_team_table.py .................                                                    [ 64%]
tests\test_sdt.py ................................................................                    [ 87%]
tests\test_token_stream.py ...............................                                           [100%]

============================================= 256 passed in 6.32s =============================================
```

Figure 11: The complete test suite run on a team member's computer (Windows, Python 3.12): 256 tests passed.

19 Results

All 256 automated tests pass, and the 42 end-to-end cases produce exactly the expected output (Figure 12). This section shows representative executions of each category and then justifies, case by case, why each input produces its result.

19.1 Representative executions

Figures 13 to 16 show one run of each category with the real program: valid inputs, syntax errors, semantic errors and lexical errors. Each run prints exactly the message predicted by the theory of the phase that stops the pipeline.

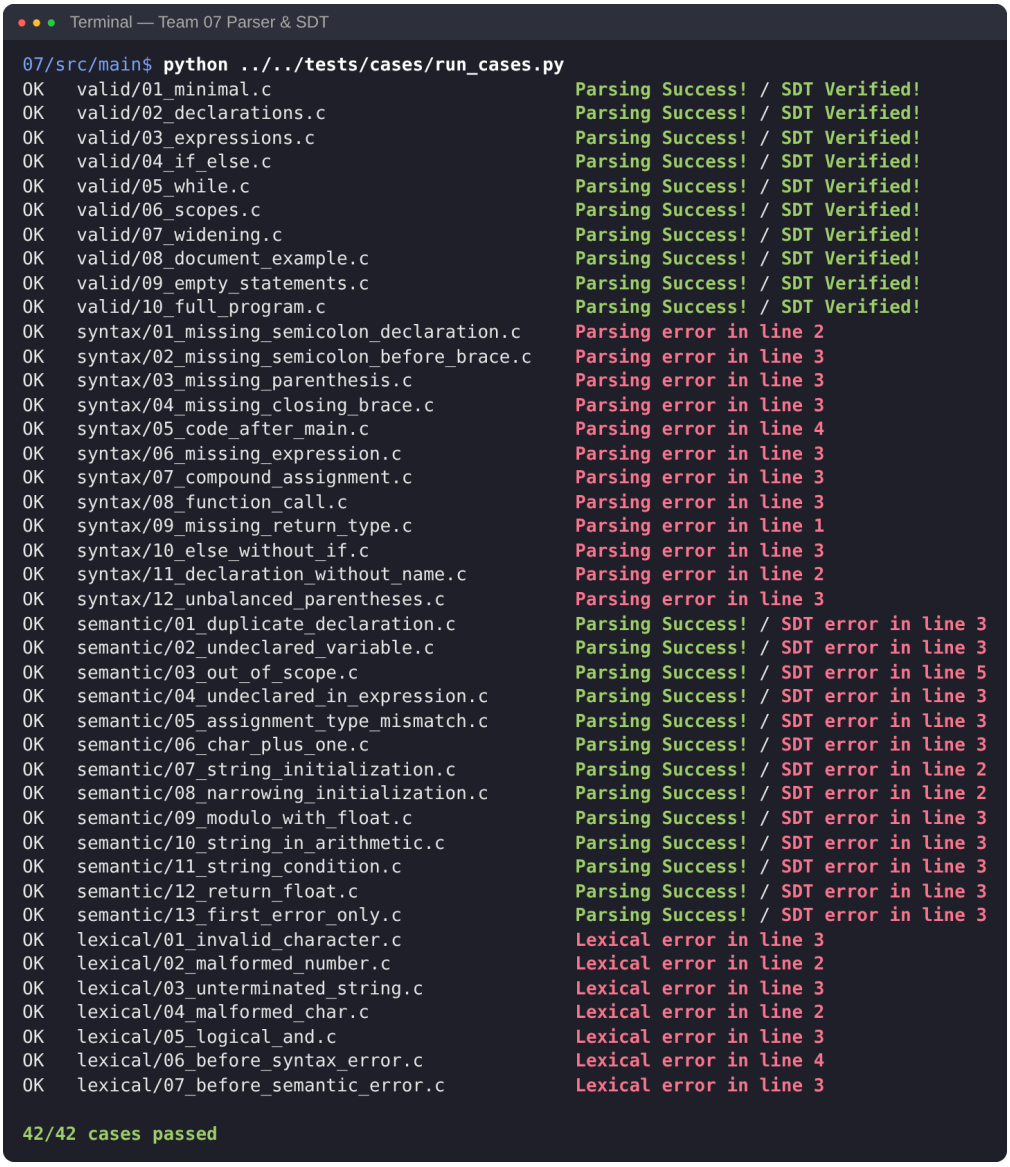


Figure 12: End-to-end cases: 42/42 produce the expected output.

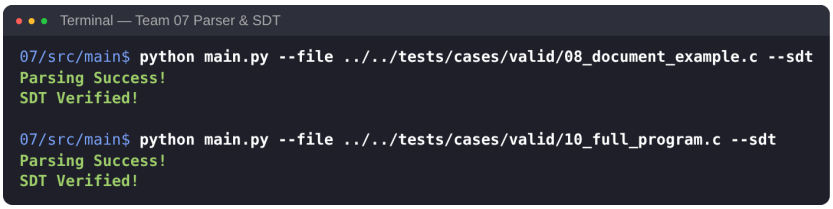


Figure 13: Valid input results.



Figure 14: Syntax error results.



Figure 15: Semantic error results (parsing succeeds, SDT fails).



Figure 16: Lexical error results: the parser is never executed.

19.2 Theoretical justification of each output

`Parsing Success!` The CLR(1) driver reached the cell ACTION[1, $] = accept. This only happens when the stack is `0 1`, that is, when the whole input has been reduced to `Program` and the next symbol is the end of input. Since the table is the deterministic automaton of the viable prefixes of the grammar, every shift and reduce taken was valid, and the sequence of reductions is a rightmost derivation of the input in reverse; hence the input belongs to $L(G)$ and its parse tree exists.

`Parsing error in line n` The driver reached an empty cell ACTION$[s, a]$. Because the CLR(1) parser has the *valid prefix property*, it detects the error at the first token a such that the prefix read so far followed by a cannot be the beginning of any sentence of the grammar. Therefore no parse tree exists and the SDT cannot be evaluated.

`SDT Verified!` The parser accepted, and every semantic action of every node of the tree was satisfied: each name was declared in a visible scope and not duplicated in its own scope, and every attribute `data_type` satisfied the type rules of its production.

`SDT error in line n` The parser accepted (so it printed `Parsing Success!`), but at least one semantic action failed. The property violated is context-sensitive or depends on attributes, so it cannot be detected by the grammar; it is detected in the tree, after parsing.

`Lexical error in line n` The input contains a character sequence that does not match any token pattern of the lexer. There is no valid sequence of terminals, so the parser is not executed.

19.3 Justification of each test case

Table 10: Valid syntax and valid semantics.

Case	Output	Why
01_minimal	Success / Verified	`Items ->` ε (p4) gives an empty block; the derivation `Program -> int main ( ) { }` exists and there is nothing to check.
02_declarations	Success / Verified	Every declaration matches p7 with or without `Init`; each initializer has the same type as its variable.
03_expressions	Success / Verified	Precedence levels p25–p50 generate every expression; all operands are numeric, `%` only uses int, and the results (int or float) are assignable to their destinations.
04_if_else	Success / Verified	`Matched`/`Unmatched` attach the inner `else` to the nearest `if` without conflicts; conditions are int.
05_while	Success / Verified	p18/p21 accept `while` with and without braces, nested; conditions are numeric comparisons.
06_scopes	Success / Verified	Each `Block` opens a scope: the inner `float x` shadows the outer `int x` legally, and the outer variables remain visible in nested blocks.
07_widening	Success / Verified	Only widening conversions (`char -> int -> float`); `'b' + 1` is int and is stored in a float.
08_document_ example	Success / Verified	Example of Section 15.1: `x + 2.5` is float, `return x` is int.
09_empty_statements	Success / Verified	`Simple -> ;` (p24) and empty blocks are valid statements; `0` and `1` are numeric conditions.
10_full_program	Success / Verified	Combines all the constructs; `suma / 5.0` is float, assigned to the float `promedio`.

Table 11: Invalid syntax.

Case	Output	Why
01_missing_ semicolon_declaration	Parsing error, line 2	After `int x = 5` the parser expects an operator or `;`; `return` (line 3) has no action. Inserting `;` would make `return` valid, so the error belongs to line 2.
02_missing_ semicolon_before_ brace	Parsing error, line 3	`return x` followed by `}`: p23 requires `;`. The `}` is on line 4, but the missing `;` is on line 3.
03_missing_ parenthesis	Parsing error, line 3	In `if (a > 0 {`, after an expression inside `if (` only an operator or `)` is valid; `{` has no action.
04_missing_closing_ brace	Parsing error, line 3	The input ends (`$`) while `Block` still needs `}`; ACTION[s, $] is empty. `$` is placed after the last token, on line 3.
05_code_after_main	Parsing error, line 4	After `Program` is complete only `$` is valid; `int` on line 4 has no action. Only one `main` function is a `Program`.
06_missing_expression	Parsing error, line 3	After `a =` an `Expr` must start; `;` is not in FIRST(Expr).

Case	Output	Why
07_compound_assignment	Parsing error, line 3	`+=` is tokenized as `+ =`; after `ID` at the start of a statement only `=` is valid.
08_function_call	Parsing error, line 3	Calls are not in the grammar: after `ID` at the start of a statement, `(` has no action.
09_missing_return_type	Parsing error, line 1	ACTION[0, t] is defined only for `int`; the first terminal is `main`.
10_else_without_if	Parsing error, line 3	`else` is not in FIRST(Item) and no open `if` can take it.
11_declaration_without_name	Parsing error, line 2	In I_{22} (`Type -> int .`) the only lookahead is `ID`; `=` has no action.
12_unbalanced_parentheses	Parsing error, line 3	The second `)` has no matching `(`: after a complete expression only `;` or an operator is valid.

Table 12: Valid syntax but invalid semantics (all print `Parsing Success!` first).

Case	Output	Why
01_duplicate_declaration	SDT error, line 3	SEM001: `x` already exists in scope 0. The grammar allows any sequence of declarations; uniqueness is context-sensitive.
02_undeclared_variable	SDT error, line 3	SEM002: `y = ...` is a valid p22, but `y` is not in any active scope.
03_out_of_scope	SDT error, line 5	SEM002: `z` was declared in scope 1, which was closed with its block.
04_undeclared_in_expression	SDT error, line 3	SEM002: p45 cannot find `c`.
05_assignment_type_mismatch	SDT error, line 3	SEM003: `2.5` is float; assignable(int, float) is false (narrowing).
06_char_plus_one	SDT error, line 3	SEM003: `c + 1` promotes char to int; int cannot be stored in char.
07_string_initialization	SDT error, line 2	SEM004: `"hello"` is string; strings are not assignable to any declarable type.
08_narrowing_initialization	SDT error, line 2	SEM004: `65` is int; int → char is narrowing under the team policy.
09_modulo_with_float	SDT error, line 3	SEM005: p39 requires char or int operands; `f` is float.
10_string_in_arithmetic	SDT error, line 3	SEM005: p34 requires numeric operands; `"dos"` is string.
11_string_condition	SDT error, line 3	SEM006: the condition of `while` has type string, not numeric.
12_return_float	SDT error, line 3	SEM007: `f` is float; `main` returns int.
13_first_error_only	SDT error, line 3	Two errors (SEM002 in line 3, SEM001 in line 4); the analysis visits in source order and stops at the first.

Table 13: Lexical errors (the parser is never executed).

Case	Output	Why
01_invalid_character	Lexical error, line 3	`@` does not belong to any token pattern.

Case	Output	Why
02_malformed_number	Lexical error, line 2	`12abc` is a single malformed token (longest match), not a constant followed by an identifier.
03_unterminated_string	Lexical error, line 3	The string has no closing quote before the end of the line.
04_malformed_char	Lexical error, line 2	A character literal must contain exactly one character; `'ab'` has two.
05_logical_and	Lexical error, line 3	`&` is not an operator of the subset, so `&&` is a lexical error.
06_before_syntax_error	Lexical error, line 4	There is a syntax error in line 2, but the lexer runs first and `#` invalidates the token stream: the parser never runs.
07_before_semantic_error	Lexical error, line 3	There is a semantic error in line 2 (`y` undeclared), but `$` in line 3 stops the process before the parser and the SDT.

20 Conclusions

The project fulfills the requirements of the statement: the input is the token stream of the team's lexer; the grammar is defined on exactly those tokens; a bottom-up CLR(1) parser builds the parse tree; and SDT rules verify it. The program distinguishes the four results required, including the case in which parsing succeeds but the SDT fails, and reports the line of each error. Everything was built from the theory seen in class, without parser generators.

Building the canonical collection showed in practice why CLR(1) is more powerful than SLR(1): the lookaheads computed with FIRST(βa) place each reduction only where it is valid in its context, and states with the same core but different lookaheads (such as I_9 and I_{122}) remain separate. As a result, a grammar written naturally, with left recursion, ε-productions, six precedence levels and the *dangling else* solved by `Matched`/`Unmatched`, has a table of 161 states with no conflicts.

The separation between syntax and semantics was also clear in the results. The grammar decides the *structure* of the program, and the SDT decides the properties that depend on context: declarations, scopes and types. The semantic test cases are syntactically correct programs that differ from valid ones only in an attribute or in a declaration located elsewhere, which is exactly why they must be detected after parsing.

Finally, keeping the grammar and the table as data shared by the documents, the code and the tests made the model and the implementation consistent, and the 256 automated tests give evidence that every output of the program has the theoretical cause described in this report.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2006.

[2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA, USA: Pearson Addison-Wesley, 2007.

[3] D. E. Knuth, "On the translation of languages from left to right," *Information and Control*, vol. 8, no. 6, pp. 607–639, 1965.

[4] Compilers, Group 5, "Parser & SDT," project statement, Faculty of Engineering, National Autonomous University of Mexico, Mexico City, Mexico, 2026.

[5] B. Yáñez Barajas (Team 07, Model Designer), "Gramática y tokens, Colección canónica LR(1), Tabla CLR(1) ACTION/GOTO, Reglas SDT y tabla de símbolos," project documents, `docs/model-designer/`, 2026.

[6] pytest-dev, "pytest documentation." Accessed: Nov. 8, 2026. [Online]. Available: https://docs.pytest.org/

[7] Team 07, "Team-07: Lexical analyzer," GitHub repository, 2026. [Online]. Available: https://github.com/HansZuniga/Team-07

[8] Team 07, "Team-07-Parser-SDT: Parser and syntax-directed translation," GitHub repository, 2026. [Online]. Available: https://github.com/HansZuniga/Team-07-Parser-SDT